KRPCLAB · BOOSTERLAND

How the booster lander works

Bringing a spent first stage back to the KSC launch pad, Super Heavy style, from a kRPC script

The booster separates at about 20-35 km, still moving downrange at several hundred metres per second. The autopilot flips it, burns back toward the pad, falls through the atmosphere steering with its body and its airbrakes, and lights the engines once more for a suicide burn that stands it upright on the pad.

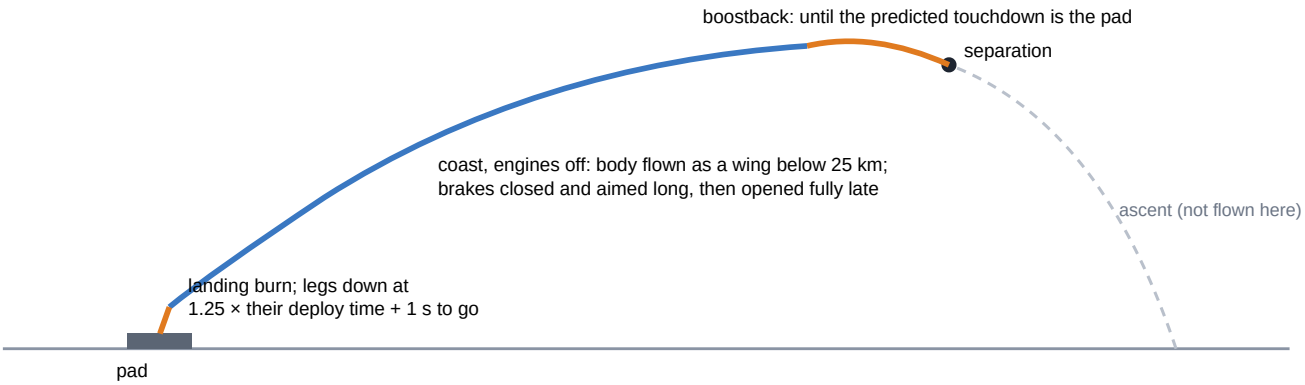


Figure 1. The flight profile, not to scale. Orange is engine-on, blue unpowered. Every part of it is re-decided each tick from a fresh prediction of where the booster will touch down.

The one idea: re-predict the landing every tick

There is no stored plan. About twenty times a second the autopilot reads the booster's position and velocity and integrates the rest of the flight to the ground in local Python: gravity, drag, body lift, the planet's rotation and the landing burn itself. Every decision that tick (where to point, how much throttle, when boostback ends, when the landing burn lights, how far the airbrakes open and at what height they open fully) is taken from that fresh prediction, so a disturbance is corrected without anything having to detect it as one.

`autoland.py` is the phase machine and the only code that talks to the game. `trajectory.py` is the propagator, `guidance.py` holds the control laws as pure functions, `engines.py` the throttle allocation, `environment.py` keeps the world data the propagator needs (density, speed of sound, drag curves, a lift slope, the holdable angle) cached so the integration makes no remote calls, and `gearcache.py` remembers how long each kind of landing leg takes to deploy.

- The prediction coasts to where the suicide burn will start and keeps integrating with the engines on, through the same throttle law the vehicle flies (`trajectory.landing_command` is shared by both).
- RK4. Drag goes as v^2 through a density that changes by a factor of e every 5 km; a first-order step would bias the touchdown.
- Drag is a curve against Mach, probed with kRPC's `simulate_aerodynamic_force_at` on the vessel as it is now, deploy angle included. The curve is re-swept every 10 s once the booster falls nose-first, and 2 s after the airbrakes move. A second curve, measured once with the brakes fully open, lets the propagator fly a brake schedule (below).
- Lift is credited only up to what the booster can hold, and discounted by the measured aoa tracking (held ÷ commanded, steady samples only).
- Heights are measured from the feet, terrain under the legs clamped against the pad's radius; the rotation vector is measured from the vessel's velocity in two frames.

The flight, phase by phase

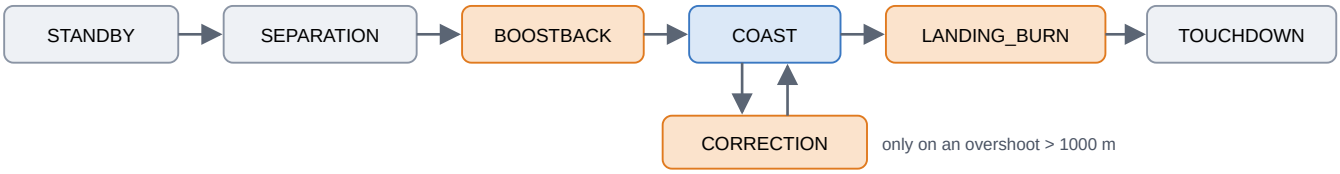


Figure 2. The phase machine. Orange phases burn the engines. CORRECTION is a short relight that COAST can enter and return from. Any phase ends the flight if the vessel is destroyed.

STANDBY waits for START	On START: RCS permitted; every airbrake deployed at 60°, the middle of the deploy law's range; every liquid engine activated and grouped for allocation; kRPC's roll threshold set to 180° so roll is controlled at any pointing error; the autopilot engaged on the current attitude.
SEPARATION a few seconds	`proximity.py` wraps each nearby craft in a sphere and marches the booster's motion forward against each, coasting and at full thrust along the boostback aim. The flip starts on the first tick nothing is in the way; only getting closer counts as a conflict; a 5 s backstop flips anyway.
BOOSTBACK flip and burn	Aim opposite the predicted horizontal miss and go to full throttle before the turn is finished, so the gimbal does the turning; the flip is two pitches through the vertical (below). While full thrust along the nose as it points now would close on a neighbour, the throttle is held at zero. Throttle tapers with the remaining Δv; the burn ends when the predicted touchdown, landing burn included, has stayed within 190 m of the pad for four ticks.
COAST engines off	Flip to the descent attitude (through the vertical again) and fall back toward the pad. On the first coast tick, a booster whose leg types are not all in the cache cycles its legs once to time them. Below 25 km the body is flown as a wing; the airbrakes first fly the late-brake schedule, then the ordinary deploy law (next section). At the predicted ignition point → LANDING_BURN. On a predicted overshoot over 1000 m with height and time left → CORRECTION, except while the brakes or the legs are being measured.
CORRECTION relight, rare	A throttled-down (≤ 40%) relight, steered within 50° of retrograde, which in the game can only shorten the flight. So it is entered only when the miss is long (or mostly cross-track), never on a short miss, and only above 1000 m. Each burn is gated on a propagated gradient (metres closed per m/s), ends under 15 m, and is cut if the miss grows past 1.3 × (or 60 m beyond) the best it has reached. At most three burns, 20 s apart.
LANDING_BURN suicide burn	On entry the engine gimbals stop answering the roll input and the RCS valve is held open for the whole burn. Throttle follows a constant-deceleration descent profile; steering is ZEM/ZEV terminal guidance on the propagator's predicted touchdown, rate-limited, and stood upright over the last 50 m (below). Legs go down by their measured time (below).
TOUCHDOWN hand back	Engines cut on reported contact (or a hover a few metres up). The situation, distance and speed are recorded 2 s later; the booster is then watched 8 s more and the sign-off adds its tilt off vertical, now and at worst (`tilt= max=`). Every exit path runs the same shutdown: throttle zero, autopilot off, SAS on, every engine limit back to 100%, gimbal roll re-enabled, panel removed.

Attitude: flips through the vertical, roll held all flight

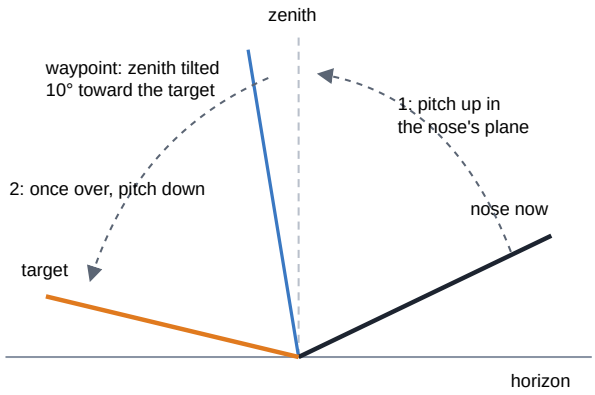


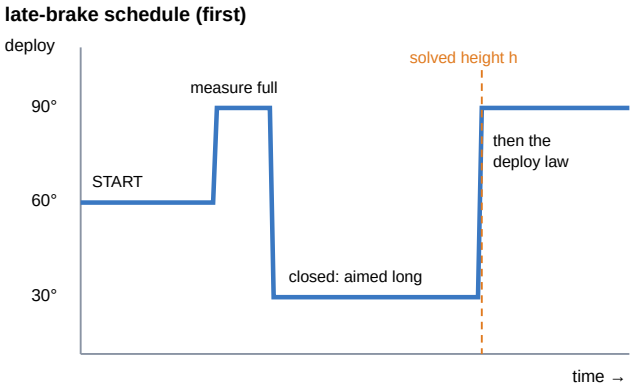
Figure 3. Any turn over 90° is two pitches through the vertical. While the nose's horizontal part points away from the target's, the command is the zenith tilted 10° toward the target; once it points the target's way, the command is the target. Memoryless: it re-reads the real nose every tick.

Roll reference. `guidance.roll_reference` puts the pitch axis normal to the trajectory plane, the plane every steering command lies in. The first roof is taken on the side of that plane nearest the roof the vehicle actually has, so it never asks for more than 90° of roll. After that each tick's roof is the side nearest the previous roof carried by the nose's own rotation (`transport_roof`): a flip over the top turns a roof that pointed up into one that points down, pitch alone, no half roll. Below 5 m/s of horizontal speed the roof is carried rather than re-taken.

Who rolls the booster. RCS and wheels always; the engine gimbals (`ModuleGimbal.enableRoll`) from START until the landing burn, then pitch and yaw only, back on at shutdown. kRPC credits a cluster's gimbals with almost no roll torque, so its roll gains would spin the booster up at ignition.

Two controls in the coast: lift and drag

Engine burns after boostback can only shorten the flight. The air can push both ways, and the coast uses it twice: the angle of attack makes a side force, and the airbrakes' deploy angle sets the drag. Drag carries the range; lift trims what is left and the cross-track.



deploy law (after the switch, or without it)

long = along-track miss of the unsteered prediction

S = Δlong / Δln(CdA) (two propagations, drag × 1.05)

k = Δln(CdA) / Δangle (curve before / after a step)

Δangle = −0.7 · long / (S·k), ≤ 20° a step, every 10 s

30-90°, deadband 30 m, re-sweep drag 2 s after each step

more open: more drag, lands shorter.

The first step is a fixed 10° that takes the first k.

Figure 4. The airbrakes. Left: once the booster is falling nose-first and its drag has been re-swept, the brakes are opened fully once to measure the full-brake drag curve, closed, and held closed so the booster flies long; the propagator flies "closed until h, fully open below", and h is the height that lands on the pad. Right: the ordinary Newton law that takes over once the full brakes are on.

Late brakes (`Autoland.run_late_deploy`). Slower at ignition is a cheaper burn, so the coast arrives as fast as it can and spends its drag at the end. The sequence waits for the coast's descent re-sweep and for the booster to be in descent attitude, so the measurement is never taken mid-flip. Then:

- brakes to 90°; 2 s later every Mach bin is probed at the real attitude (`Environment.sweep_bins`) and stored as the full-brake curve;
- brakes to 30°, and a re-sweep that *replaces* the current curve with the closed-brake one. If the full curve has no more drag than the closed one (median ratio ≤ 1), the schedule is abandoned and the ordinary law flies;
- every 5 s, a bisection on the unsteered prediction's along-track miss finds the switch height h, to 10 m. Long even opening now: open now. Short even never opening: never. The environment holds h (`late_switch_alt`), so *every* prediction, from the lift solve to the burn trigger, flies the schedule;
- at h, in COAST, CORRECTION or the burn, the brakes go to 90° and the propagator uses the full curve from then on; once the replacing re-sweep of the open brakes has been taken, the schedule is handed to the ordinary law.

Deploy law (`Autoland.run_deploy`). A Newton step every 10 s on a gain it works out for itself, so no constant belongs to one craft. The miss it answers is the along-track miss of the unsteered prediction: the lift solve re-nulls the steered prediction every tick, so judged on that the brakes would see ~0 until the lift saturated.

The holdable angle. Every 2 s `Environment.refresh_holdable` asks the game how hard the air pushes back: `simulate_aerodynamic_torque_at` on the vessel turned a few degrees off retrograde, reduced to the restoring moment per pascal and stored by Mach. Against it stands the control torque: wheels plus the highest RCS torque seen plus control surfaces scaled by q, times 0.7. Where they meet is the most the booster can hold, typically 10° at low q falling to ~1° at 50 kPa.

Lift (below 25 km). The angle is solved each tick: propagate small test angles in-plane and out-of-plane, invert the 2×2 to null downrange and cross-track, then propagate the answer and keep it only if it lands nearer than doing nothing (else halve once, else hold retrograde). The command is capped at the holdable angle and 10°.

The landing burn: profile, steering, standing up

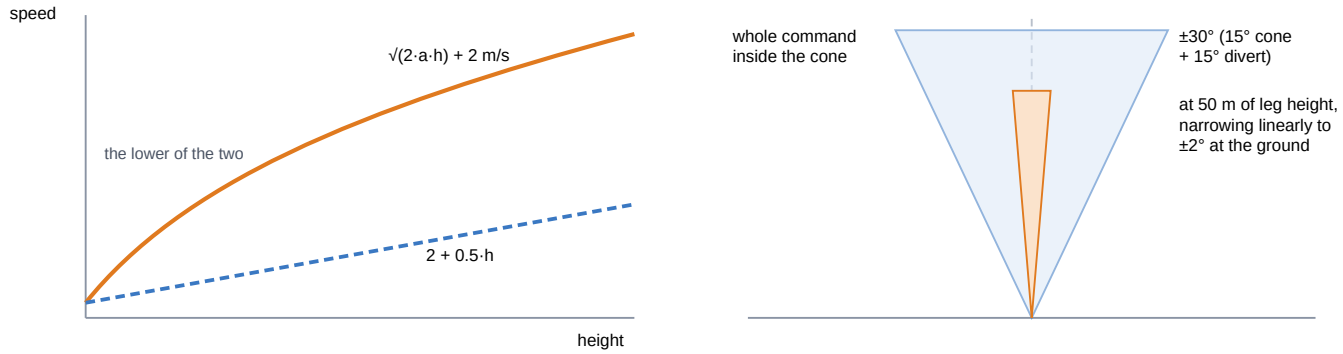


Figure 5. Left: the throttle tracks the lower of two curves, a constant-deceleration stop at 95% of available thrust blending into a constant-rate final descent. Right: below 50 m of leg height the whole steering command, base and divert, is held inside a cone round vertical that narrows to 2° at the ground, so the booster touches down standing.

Trigger and throttle. The burn lights when the height falls to the full-throttle stopping distance × 1.18 plus 1.5 s of fall. Feed-forward plus a trim of 6 m/s² per m/s of error, divided by the booster's own maximum acceleration, so a 9 g and a 3.7 g booster get the same loop. The tracked speed is √(v_down² + (w·v_side)²), w = height ÷ 250 m (0-1): above 250 m all of |v| is braked, near the ground only the descent rate, sideways speed being the divert's job. Below 60 m an ascending booster gets zero throttle; below 12 m at under 8 m/s the throttle is capped at hover plus 0.12.

Steering. ZEM/ZEV guidance, gains 6 and 4, aimed at the propagator's predicted touchdown. The tilt is capped at 15° and at acos(throttle ÷ cap), the angle the spare throttle can pay for; the throttle is opened by 1/cos(tilt). Below 250 m the position term is dropped and only the sideways velocity is nulled, from a base of retrograde held inside 15° of vertical (vertical when ascending). **Standing up** (`guidance.terminal_command`): below 50 m of leg height the command is then clamped into a cone round vertical that shrinks linearly from 30° to 2° at the ground; retrograde swings toward the horizontal as the descent stops, and without the clamp any sideways speed left would become a lean at contact.

Command rate. The command may move no faster than a quarter of a second of the vehicle's own angular acceleration per second: kRPC's credited pitch and yaw torque over the moment of inertia, the weaker axis, measured per unit throttle once a game second while the engines are lit and scaled by this tick's throttle, never under 5°/s. Before the first torque reading the command is not limited. The RCS valve is held open through the burn, which keeps kRPC's auto-tuned gains steady (they follow the torque it believes is available).

Landing legs: timed once, dropped by time to go



Figure 6. kRPC exposes neither a group's membership nor a part's animation length, so the legs are timed in flight, once per part name, and kept in `boosterland/data/gearTimes.json`.

On the first COAST tick the autopilot lists the vessel's deployable legs and wheels. If every part name is in the cache, the slowest cached deploy time is used. Otherwise it lowers the gear, records for each part the time to reach `deployed` (a broken part is not timed), raises it again, times the retraction and merges the results into the cache (a file lock and an atomic replace, so farm instances can share it); a cycle not done in 20 s leaves the legs untimed. With a time in hand the gear goes down in COAST, CORRECTION or the burn as soon as the time to go (the smaller of height ÷ descent rate and the prediction's time to land) is under 1.25 × the deploy time + 1 s: for the LT-2 leg, 4.6 s, that is 6.75 s before the ground. Without one (no legs, or timing failed) the gear goes down at the burn's first tick, with a 350 m backstop.

Engine allocation: one group at a time

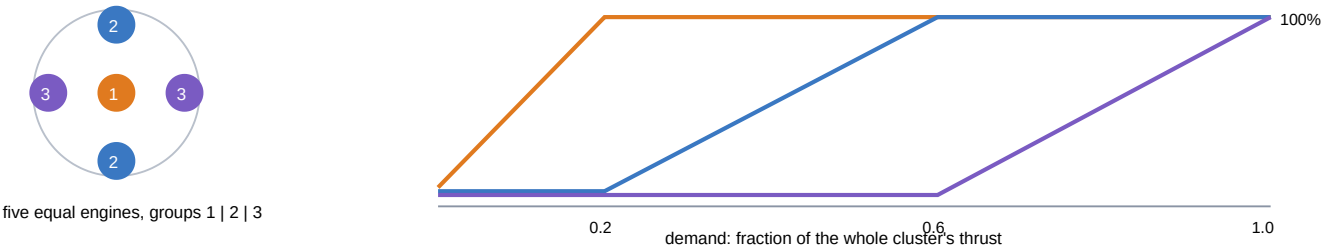


Figure 7. Each group's thrust limit against the demand. A group stays dark until the group before it is at 100%; only the group in progress is part-throttled. The main throttle is 1 whenever demand is above 0.

At START, `EngineCluster` reads each liquid engine's position in the vessel frame and its thrust, and `engines.group_engines` groups them about the thrust-weighted centroid: within 0.3 m of the axis is the center group; in a ring an engine is paired with the one whose offset most nearly cancels its own; whatever cannot be paired lights together, so every group's thrust passes through the axis. Guidance reads `max_thrust`, which ignores the limits, so the planner always sees the whole cluster. A single-engine booster is one group.

Smaller rules

- RCS: in COAST the valve opens only when the attitude controller has been saturated (input ≥ 0.95 for 0.5 s) and is not closing; in the landing burn it is held open; elsewhere it is a relay on pointing error.
- A destroyed vessel ends the flight. Its streams freeze rather than fail, so a position bit-identical for 1 game-second while moving faster than 1 m/s is a vessel lost. A mass drop of more than 10% in one tick is logged as parts lost.
- Every telemetry line carries the commanded nose's angle off retrograde (`cmd=`) and off vertical (`cv=`), how fast the command moves (`cw=`), the pointing error (`err=`), the largest control input (`sat=`) and each input (`in=`), the roll error (`roll=`), the nose's swing and roll rates (`w=`, `r=`), each engine group's delivered thrust (`eng=`), the angle held (`hold=`) and the holdable angle (`can=`). The events say when the legs were timed (`gear timed:`, or `(cached)`), when they went down and why, and each late-brake step (`late brakes: open at h long=…`, `late brakes 30 -> 90 at H m`).
- Every log ends with the loop rate per phase, and the config line carries a fingerprint of the defaults. A commanded throttle the vessel does not report is logged: a physical throttle axis winning.

How it is measured

Each flight writes a numbered log (`logs/LOG<n>`), telemetry paced in game time, with the settings that differ from the defaults. Tunables live only in `config.py`; `--set FIELD=VALUE` overrides one for a flight. Reference saves (`quicksave`, `qs_hot`, `qs_cold`, `qs_steep`, `qs_north` for the 17 t single-engine booster, which has no legs; `qs_heavy` for the 99 t five-engine booster on six LT-2 legs) give the separation states; `tools/savegen.py` writes more, and `tests/fakeksp` runs the closed loop offline for screening. `tools/brot.sh` flies arms × saves rotated over the farm at 1×.

State of the code on 2026-10-18 (defaults 9cba676e). Tunables: `boosterland/config.py`.